DOCUMENTACIÓN TÉCNICA — ServiDental

Proyecto: ServiDental CR — sitio web y tienda en línea de equipo médico dental (Costa Rica) **Repositorio:** `servidental-next` **Producción:** https://servidentalcr.com **Backend de contenido y catálogo:** https://wp.servidentalcr.com (WordPress + WooCommerce) **Idioma del sitio:** Español (Costa Rica) · **Moneda:** colón (₡) y dólar ($)

Tabla de contenido

1. Resumen del proyecto
2. Arquitectura
3. Estructura del repositorio
4. Puesta en marcha local
5. Variables de entorno
6. Dependencias
7. Módulos funcionales clave
8. Convenciones de código
9. Guía de Clean Code
10. Checklist de despliegue
11. Buenas prácticas de Handoff
12. Guía de documentación técnica en Markdown
13. Deuda técnica y riesgos conocidos
14. Resolución de problemas frecuentes

---

1. Resumen del proyecto

ServiDental CR es una aplicación **Next.js 15 (App Router)** que combina páginas informativas, un blog y una tienda en línea completa para profesionales dentales costarricenses.

Capacidad	Implementación
Catálogo y pedidos	WooCommerce (REST API v3) en `wp.servidentalcr.com`, consumido a través de rutas API propias de Next.js
Blog	WordPress como CMS headless (REST API `wp/v2`)
Pagos con tarjeta	TiloPay (SDK en `/checkout` y rutas `api/tilopay/*`)
Pago por transferencia	Flujo con descuento promocional configurable por fecha
Carrito	Contexto de React (`CartProvider`) con persistencia en el navegador
Cotizaciones	Productos con etiqueta `cotizar` se gestionan por WhatsApp / formulario
Formularios de contacto	EmailJS

Capacidad	Implementación
Analítica	Google Analytics (`GoogleAnalytics.tsx` , `lib/analytics.ts`)
Despliegue	Vercel

> **Nota sobre documentación previa:** `CLAUDE.md` y partes de `README.md` aún mencionan ONVO como procesador de pagos. El proveedor vigente en el código es **TiloPay**. Este documento refleja el estado real del código.

---

2. Arquitectura

2.1 Visión general

```
Navegador
   │
   ▼
Next.js 15 (Vercel)
   ├── Páginas (App Router, Server + Client Components)
   ├── Rutas API  /api/woocommerce/*  ──►  WooCommerce REST API  (credenciales solo en servidor)
   ├── Rutas API  /api/tilopay/*      ──►  TiloPay API
   ├── Rutas API  /api/wordpress/*    ──►  WordPress REST API
   └── Rutas API  /api/reviews/*      ──►  Reseñas de producto
```

2.2 Principios de diseño

- **Patrón proxy:** el navegador nunca habla directo con WooCommerce ni con TiloPay. Las rutas `/api/*` añaden credenciales del lado servidor y evitan problemas de CORS.
- **Estado del carrito en cliente:** `CartProvider` (`src/hooks/useCart.tsx`) envuelve toda la app desde `src/app/layout.tsx` .
- **Renderizado híbrido:** contenido estático donde es posible; rutas dinámicas (`force-dynamic`) para blog y datos de tienda que cambian con frecuencia.
- **Imágenes:** `images.unoptimized: true` en `next.config.js` para evitar el límite de optimizaciones del plan Hobby de Vercel (error 402). Por eso los activos locales deben subirse **ya optimizados** (AVIF/WebP).
- **Redirecciones:** `/products` y `/products/*` redirigen de forma permanente a `/tienda` (SEO).

2.3 Flujo de compra

1. El usuario navega `/tienda` y `/tienda/[slug]` (datos desde `/api/woocommerce/products`).
2. Agrega al carrito (`useCart` , validaciones en `utils/stock.ts`).
3. En `/checkout` completa sus datos y elige método de pago.
4. Se crea el pedido en WooCommerce (`/api/woocommerce/orders`).
5. Tarjeta: se crea el pago en TiloPay y se redirige/confirma vía `/checkout/tilopay-callback` ; `api/tilopay/notify` y `api/woocommerce/orders/update-payment` actualizan el estado del pedido.
6. Resultado: `/checkout/success` , `/checkout/failure` o `/checkout/cancel` .

---

3. Estructura del repositorio

```
servidental-next/
├── middleware.ts              # Excluye estáticos (manifest, favicons, imágenes) de cualquier auth/preview
├── next.config.js             # Imágenes, redirects, env públicas, regla webpack para video
├── tailwind.config.ts         # Tema: servi_green, servi_dark, servi_light, gradientes
├── public/                    # Favicons, og-image.jpg, site.webmanifest, robots.txt, sitemap.xml, badges,
videos
├── docs/                      # Documentación técnica (este documento)
├── backlog/                   # Planes y diagnósticos pendientes (SEO, TiloPay)
├── migrations/                # Scripts de migración de productos (solo referencia, ya ejecutada)
├── scripts/                   # Utilidades de desarrollo
└── src/
    ├── app/                   # Rutas (App Router)
    │   ├── api/               # woocommerce, tilopay, wordpress, reviews
    │   ├── tienda/            # Listado, producto [slug], marca [slug]
    │   ├── checkout/          # Checkout y páginas de resultado
    │   ├── blog/              # Listado y detalle
    │   └── (about, contact, services, spare-parts, privacy-policy, terms-conditions, x-ray-certification)
    ├── components/
    │   ├── ecommerce/         # cart, checkout, filters, product, quote, ui
    │   ├── home/              # Hero, secciones y banners de campaña
    │   ├── blog/ contact/ common/ layout/ products/ spareParts/
    ├── hooks/                 # useCart, useWooCommerce
    ├── lib/                   # tilopay.ts, analytics.ts
    ├── server/                # wc.ts (cliente WooCommerce), brands.ts, sanitize.ts, productResources.ts
    ├── services/              # blogService.ts, producService.ts
    ├── types/                 # Tipos de WooCommerce, blog, producto, repuestos
    ├── utils/                 # currency, promo, delivery, stock, whatsapp, woocommerce, brandLogos
    ├── data/                  # Datos manuales (productos, repuestos)
    └── assets/                # Imágenes locales (AVIF/WebP)
```

Regla de ubicación: la lógica de negocio pura vive en `utils/`; el acceso a servicios externos en `server/`, `services/` o `lib/`; la UI en `components/`; las páginas solo componen.

---

4. Puesta en marcha local

Requisitos: Node.js 20 LTS o superior, npm, acceso a las credenciales de WooCommerce (REST API).

```
# 1. Instalar dependencias
npm install

# 2. Crear el archivo de entorno
cp .env.example .env.local     # y completar los valores (ver sección 5)

# 3. Levantar el servidor de desarrollo
npm run dev                    # http://localhost:3000
```

Scripts disponibles

Comando	Descripción
`npm run dev`	Servidor de desarrollo
`npm run dev:servidental`	Desarrollo cargando `.env.servidental.local`
`npm run dev:aurigital`	Desarrollo cargando `.env.aurigital.local`
`npm run build`	Build de producción
`npm run start`	Sirve el build de producción
`npm run lint`	Linter de Next.js
`npm run parse-products`	Convierte productos TypeScript a JSON

---

5. Variables de entorno

Copiar `.env.example` a `.env.local`. **Nunca** subir archivos `.env*` con valores reales al repositorio. En Vercel se configuran en *Project Settings → Environment Variables*.

Variable	Alcance	Obligatoria	Uso
`WOOCOMMERCE_URL`	Servidor	Sí	URL base de WooCommerce, sin barra final
`WOOCOMMERCE_CONSUMER_KEY`	Servidor	Sí	Clave de la REST API
`WOOCOMMERCE_CONSUMER_SECRET`	Servidor	Sí	Secreto de la REST API
`WORDPRESS_BASE_URL`	Servidor	Sí	URL base de WordPress (blog)
`WORDPRESS_API_URL`	Servidor	Sí	Endpoint `wp/v2`
`NEXT_PUBLIC_BASE_URL`	Público	Sí (preview)	URL pública del sitio para fetch en SSR
`TILOPAY_API_USER` / `TILOPAY_API_PASS` / `TILOPAY_API_KEY`	Servidor	Para pagos	Credenciales de TiloPay
`TILOPAY_API_BASE`, `TILOPAY_AUTH_PATH`, `TILOPAY_PAYMENT_PATH`	Servidor	Opcional	Endpoints de TiloPay

Variable	Alcance	Obligatoria	Uso
`TILOPAY_CALLBACK_URL`, `NEXT_PUBLIC_TILOPAY_REDIRECT`	Servidor / Público	Para pagos	URL de retorno tras el pago
`TILOPAY_AMOUNT_UNIT`	Servidor	Opcional	Unidad de monto enviada a TiloPay
`EMAILJS_SERVICE_ID` / `EMAILJS_TEMPLATE_ID` / `EMAILJS_USER_ID`	—	Formularios	Envío de correos de contacto
`NEXT_PUBLIC_WOOCOMMERCE_URL`	Público	Opcional	URL pública de WooCommerce

Reglas:

- Todo lo que lleve `NEXT_PUBLIC_` queda **expuesto en el navegador**. Jamás usarlo para secretos.
- Validar variables obligatorias al inicio de cada ruta API y responder con error claro (ver `src/server/wc.ts`).
- Mantener `.env.example` sincronizado: si se agrega una variable al código, se agrega al ejemplo en el mismo commit.

> Ver riesgos abiertos sobre variables públicas con credenciales en la sección 13.

---

6. Dependencias

6.1 Producción

Paquete	Versión	Propósito
`next`	^15.4.8	Framework (App Router)
`react` / `react-dom`	18.3.1	UI (versión fijada)
`@woocommerce/woocommerce-rest-api`	^1.0.2	Cliente oficial WooCommerce
`woocommerce-api`	^1.5.0	Cliente legado (candidato a retirarse)
`axios`	^1.10.0	Cliente HTTP
`@emailjs/browser`	^4.4.1	Correos desde formularios
`@headlessui/react`	^2.1.10	Componentes accesibles sin estilo
`@heroicons/react`, `lucide-react`, `react-icons`	^2.1.5 / ^0.453.0 / ^5.3.0	Iconografía
`framer-motion`	^11.11.9	Animaciones
`react-virtuoso`	^4.18.1	Listas virtualizadas
`yet-another-react-lightbox`	^3.21.7	Galería de imágenes
`sanitize-html`	^2.17.0	Sanitizado de HTML proveniente de WordPress
`sharp`	^0.33.5	Procesamiento de imágenes
`dotenv`	^17.2.0	Carga de variables en scripts

6.2 Desarrollo

Paquete	Versión	Propósito
`typescript`	^5	Tipado
`tailwindcss` / `postcss`	^3.4.1 / ^8	Estilos
`@types/*`	varias	Tipos de Node, React, jsdom, sanitize-html
`ts-node` , `tsx`	^10.9.2 / ^4.20.3	Ejecución de scripts TypeScript
`dotenv-cli`	^10.0.0	Scripts `dev:*` con archivos de entorno por cliente
`jsdom`	^25.0.1	Utilidades de scripts

6.3 Política de dependencias

- Antes de agregar un paquete: verificar que no exista ya algo equivalente en el proyecto (hay tres librerías de iconos; no sumar una cuarta).
- Preferir versiones con mantenimiento activo y tipos incluidos.
- Actualizar con `npm outdated` y revisar changelogs; las actualizaciones mayores (Next, React, Tailwind) van en su propio commit/PR con prueba de humo completa.
- Ejecutar `npm audit` antes de cada release y atender vulnerabilidades altas/críticas.
- Commitear siempre `package-lock.json` .

---

7. Módulos funcionales clave

7.1 Promociones (`src/utils/promo.ts`)

Sistema configurable por fecha. Cada promo del arreglo `PROMOS` define etiqueta de WooCommerce que la activa, ventana de vigencia (hora de Costa Rica, UTC-6), tasa de descuento (solo por transferencia), clases del badge y nota informativa. **Activar o retirar una campaña = editar el arreglo**; se enciende y apaga sola por fecha. Las campañas pasadas deben retirarse o dejarse vencidas, nunca con fechas ambiguas.

7.2 Badges de entrega (`src/utils/delivery.ts`)

Mapa `etiqueta → días de entrega` . Agregar una entrada habilita el badge en grilla y ficha de producto.

7.3 Cotizaciones y WhatsApp (`src/utils/whatsapp.ts`)

Los productos con etiqueta `cotizar` no se compran directo: se genera mensaje de WhatsApp o se abre `QuoteFormModal` .

7.4 Carrito y cupones (`src/hooks/useCart.tsx` , `CouponInput.tsx`)

Carrito en contexto con validación de stock, cantidades y cupones (`/api/woocommerce/coupons/[code]`).

7.5 Blog (`src/services/blogService.ts` , `src/app/blog`)

Lectura desde WordPress con caché en memoria (TTL de 15 minutos). El detalle `/blog/[slug]` es Server Component; el HTML externo se sanea con `sanitize-html` (`src/server/sanitize.ts`).

7.6 Pagos TiloPay (`src/lib/tilopay.ts`, `src/app/api/tilopay/*`)

El SDK se carga **solo en `/checkout`** con `next/script` (`afterInteractive`). No volver a cargarlo globalmente en `layout.tsx`. Documentación adicional: `TiloPayImplementation.md` y `backlog/tilopay-doble-formulario-diagnostico.md`.

7.7 SEO e identidad

Metadatos y `icons` declarados en `src/app/layout.tsx`; imagen Open Graph en `public/og-image.jpg` (1200×700); favicons y `site.webmanifest` en `public/`. Plan de mejoras: `backlog/SEO-mejoras-plan.md`.

---

8. Convenciones de código

8.1 Idioma

Elemento	Idioma
Interfaz, textos al usuario, mensajes de error visibles	Español (Costa Rica)
Identificadores (variables, funciones, tipos, archivos)	Inglés
Comentarios	Español, explicando el **porqué**
Commits	Español, en imperativo

8.2 Nombres

Elemento	Convención	Ejemplo
Componentes React	`PascalCase`, un componente por archivo, archivo con el mismo nombre	`ProductCard.tsx`
Hooks	`camelCase` con prefijo `use`	`useCart.tsx`
Utilidades y servicios	`camelCase`	`getDeliveryDaysForTags`
Tipos e interfaces	`PascalCase`	`WooCommerceProduct`
Constantes globales	`UPPER_SNAKE_CASE`	`PROMOS`, `DELIVERY_BADGES`
Rutas (carpetas de `app/`)	`kebab-case`	`privacy-policy`
Rutas API	`route.ts` dentro de carpeta por recurso	`api/woocommerce/products/route.ts`
Booleanos	Prefijo `is`, `has`, `can`, `should`	`isLoading`, `canAddToCart`

8.3 TypeScript

- Usar `@/` para imports desde `src/` (alias en `tsconfig.json`); evitar rutas relativas largas (`../../..`).

- Tipar props con `interface`; evitar `any`. Para respuestas externas, definir tipo en `src/types/`.
- Usar `import type` cuando solo se importan tipos.
- Preferir funciones puras y retornos explícitos en utilidades.

8.4 React / Next.js

- Componentes **Server por defecto**; agregar `'use client'` solo cuando se necesiten estado, efectos o eventos, y lo más abajo posible en el árbol.
- Las páginas (`page.tsx`) componen; la lógica y la UI reutilizable van en `components/`.
- Acceso a datos externos solo desde servidor (`server/`, rutas API) o desde hooks dedicados; nunca `fetch` disperso dentro de componentes de presentación.
- Siempre implementar estados de **carga**, **error** y **vacío**.
- Usar `next/image` y `next/link`; todas las imágenes con `alt` descriptivo.
- Imágenes locales en AVIF/WebP, optimizadas **antes** de subirlas (la optimización de Next está desactivada).

8.5 Estilos (Tailwind CSS)

- Mobile-first: estilos base para móvil y `sm:`, `md:`, `lg:` para ampliar.
- Usar los colores del tema (`servi_green`, `servi_dark`, `servi_light`) en vez de hex sueltos.
- Si una combinación de clases se repite 3 o más veces, extraerla a un componente.
- CSS global solo en `globals.css` y `styles/wp-gutenberg.css` (contenido de WordPress).

8.6 Rutas API

- Validar variables de entorno y parámetros al inicio.
- Códigos de estado correctos: `400` entrada inválida, `404` no encontrado, `500` error interno, `502` cuando falla el servicio externo.
- Timeout explícito en llamadas externas (existe 10–12 s en el cliente WooCommerce).
- Logs con prefijo de módulo (`[WooCommerce API]`) y **sin datos sensibles**.
- Nunca devolver al cliente mensajes crudos de error del proveedor.

8.7 Git

- Ramas cortas por tarea, con prefijos: `feature/…`, `fix/…`, `docs/…`, `chore/…`.
- Commits pequeños, un propósito cada uno, mensaje en imperativo: `Corrige cálculo de descuento por transferencia`.
- No commitear `.env*`, `node_modules`, `.next`, archivos temporales ni volcados de datos con información de clientes.
- Todo cambio a `main` pasa por revisión (PR) y build exitoso.

---

9. Guía de Clean Code

Principios aplicados a este proyecto, con ejemplos del dominio.

9.1 Nombres que revelan intención

```
// ❌ Evitar
const d = getD(p.tags);
// ✅ Preferir
const deliveryDays = getDeliveryDaysForTags(product.tags);
```

9.2 Funciones pequeñas, un solo propósito

Una función hace una cosa y se puede describir sin usar "y". Si supera ~30 líneas o mezcla cálculo, formato y efectos, dividirla. La lógica de precios, promos y stock vive en `utils/`, separada de la UI.

9.3 Sin números ni textos mágicos

```
// ❌
if (total > 0.2 * subtotal) …
// ✅
const TRANSFER_DISCOUNT_RATE = 0.20;
```

Los datos de campañas, teléfonos y URLs se centralizan en constantes o configuración, no se repiten dentro de componentes.

9.4 DRY con criterio

Eliminar duplicación de **conocimiento**, no de apariencia. Hoy existen dos conjuntos de componentes de producto (`components/products/` y `components/ecommerce/product/`): consolidar antes de agregar un tercero.

9.5 Comentarios que explican el porqué

```
// ✅ Hora CR (UTC-6): la promo termina el 30-sep inclusive, por eso end es el 1-oct 00:00.
```

Código obsoleto se **elimina**, no se comenta: para eso está Git.

9.6 Manejo de errores explícito

- No tragar errores con `catch {}` vacío.
- Fallar temprano con mensajes útiles; mostrar al usuario un mensaje amable en español y registrar el detalle técnico en el servidor.

9.7 Tipado estricto

Evitar `any`. Tipar las respuestas de WooCommerce/WordPress en `src/types/`; si el tipo no existe, crearlo en lugar de castear.

9.8 Separación de responsabilidades

```
Página → compone
Componente → presenta
Hook → estado e interacción con el cliente
Servicio / server → habla con APIs externas
Utilidad → reglas de negocio puras
```

9.9 Regla del boy scout

Deja cada archivo un poco mejor de como lo encontraste: renombra algo confuso, elimina un import sin uso, tipa un `any`. Sin mezclar refactors grandes con cambios funcionales en el mismo commit.

9.10 Checklist de revisión de código

- [ ] ¿Los nombres se entienden sin leer la implementación?
- [ ] ¿Funciones cortas y de un solo propósito?
- [ ] ¿Sin `any`, `console.log` ni código comentado?
- [ ] ¿Sin valores mágicos ni secretos en el código?
- [ ] ¿Estados de carga, error y vacío cubiertos?
- [ ] ¿Responsive probado en móvil y escritorio?
- [ ] ¿Imágenes optimizadas y con `alt`?
- [ ] ¿`'use client'` solo donde es necesario?
- [ ] ¿`.env.example` y documentación actualizados si aplica?
- [ ] ¿`npm run build` pasa sin errores?

---

10. Checklist de despliegue

10.1 Antes de desplegar

- [ ] `npm run lint` sin errores.
- [ ] `npm run build` exitoso en local (no confiar en que Vercel lo "arregle").
- [ ] Revisar errores de TypeScript manualmente: el build tiene `ignoreBuildErrors: true`, por lo que **no** los detendrá. Ejecutar `npx tsc --noEmit`.
- [ ] `npm audit` sin vulnerabilidades altas/críticas.
- [ ] Variables de entorno nuevas agregadas en Vercel (Preview y Production) y en `.env.example`.
- [ ] Sin archivos `.env*`, credenciales o datos de clientes en el commit.
- [ ] Imágenes nuevas optimizadas (AVIF/WebP).
- [ ] Campañas/promos: fechas de `PROMOS` verificadas (hora CR).
- [ ] Cambios revisados mediante PR.

10.2 Vercel — Preview

- [ ] Deploy de Preview generado sin errores de build.
- [ ] `NEXT_PUBLIC_BASE_URL` configurada (o vacía para que se infiera).
- [ ] Credenciales de WooCommerce presentes en el entorno Preview.

- [ ] Credenciales de TiloPay en modo **pruebas** para Preview.

10.3 Pruebas de humo (Preview y Producción)

#	Prueba	Resultado esperado
1	`/` carga	Hero, categorías, productos y marcas visibles
2	`/tienda`	Productos y filtros cargan; sin 500/502 en consola
3	`/tienda/[slug]` de un producto	Galería, precio, variaciones, badges correctos
4	Agregar al carrito y cambiar cantidad	MiniCart y `/carrito` actualizan totales
5	Aplicar un cupón válido y uno inválido	Descuento aplicado / mensaje de error claro
6	`/checkout` (SDK TiloPay)	SDK carga solo aquí; sin 404 de scripts
7	Pago de prueba con tarjeta	Redirección a success y pedido en WooCommerce con estado correcto
8	Pago fallido / cancelado	Páginas `failure` / `cancel` correctas
9	Pago por transferencia con promo vigente	Descuento aplicado solo a productos con la etiqueta
10	`/blog` y `/blog/[slug]`	Listado y detalle cargan
11	Formulario de contacto	Correo recibido
12	`/site.webmanifest`, `/favicon.ico`, `/robots.txt`, `/sitemap.xml`, `/og-image.jpg`	Responden 200 sin autenticación
13	`/products`	Redirige a `/tienda` (301/308)
14	Vista móvil	Header, carrito, checkout y botón de WhatsApp funcionales

10.4 Producción

- [ ] Credenciales de TiloPay en modo **producción** y callbacks apuntando al dominio real.
- [ ] Dominio y HTTPS verificados; `NEXT_PUBLIC_BASE_URL` = `https://servidentalcr.com`.
- [ ] Open Graph validado (Facebook Sharing Debugger / LinkedIn Post Inspector); caché de previews actualizada si cambió la imagen.
- [ ] Google Analytics recibiendo eventos.
- [ ] Un pedido real de bajo monto de punta a punta, y reembolso si corresponde.
- [ ] Revisar logs de Vercel durante los primeros 30 minutos.

10.5 Plan de reversión

1. En Vercel → *Deployments*, promover el despliegue estable anterior (*Instant Rollback*).
2. Si el problema vino de una variable de entorno, restaurar el valor previo y volver a desplegar.
3. Documentar el incidente: qué pasó, impacto, causa y acción correctiva.

---

11. Buenas prácticas de Handoff

El objetivo: que una persona nueva pueda **entender, ejecutar y desplegar** el proyecto sin depender de quien lo entrega.

11.1 Qué debe entregarse

Acceso (por canal seguro, nunca por chat o correo en texto plano):

- [ ] Repositorio Git (permisos de escritura).
- [ ] Proyecto de Vercel y variables de entorno.
- [ ] WordPress / WooCommerce (administrador) y generación de claves REST.
- [ ] Panel de TiloPay (credenciales de pruebas y producción).
- [ ] EmailJS, Google Analytics / Search Console.
- [ ] DNS / dominio y hosting del backend.
- [ ] Gestor de contraseñas compartido con el inventario de cuentas.

Conocimiento:

- [ ] Este documento, actualizado.
- [ ] Mapa de integraciones externas y quién es dueño de cada cuenta.
- [ ] Lista de deuda técnica y riesgos (sección 13).
- [ ] Backlog priorizado (`backlog/`).
- [ ] Decisiones importantes y su motivo (ver 11.3).
- [ ] Procedimientos recurrentes: cómo montar una promo, agregar un producto, publicar en el blog, rotar credenciales.

11.2 Proceso de entrega

1. **Congelar** el alcance: lista de lo terminado, en curso y pendiente.
2. **Limpiar**: ramas obsoletas, archivos temporales, scripts muertos, TODOs resueltos.
3. **Verificar** que el proyecto levanta desde cero en una máquina limpia: clonar → `npm install` → `.env.local` → `npm run dev` → `npm run build` .
4. **Sesión de traspaso** (60–90 min, grabada si es posible): recorrido por arquitectura, flujo de compra, despliegue y puntos delicados.
5. **Periodo de acompañamiento:** ventana acordada donde quien entrega responde dudas.
6. **Confirmación de recibido** por escrito y rotación de credenciales que ya no deban conservarse.

11.3 Registro de decisiones (ADR ligero)

Cada decisión técnica relevante se registra con este formato breve:

```
### ADR-001: Desactivar la optimización de imágenes de Vercel
- **Fecha:** 2025-xx-xx
- **Contexto:** El plan Hobby limita a 1000 optimizaciones/mes; se excedió y devolvió error 402.
- **Decisión:** `images.unoptimized: true`; las imágenes se suben ya optimizadas.
- **Consecuencias:** Más trabajo manual de optimización; sin costos por límite.
```

11.4 Señales de un buen handoff

- Quien recibe despliega a Preview **sin ayuda** el primer día.
- No existen secretos solo en la cabeza de una persona.
- Los problemas conocidos están escritos, no "se sabe".
- El historial de commits cuenta una historia entendible.

---

12. Guía de documentación técnica en Markdown

12.1 Dónde vive cada cosa

Documento	Ubicación	Contenido
Visión general y arranque rápido	`README.md`	Qué es, cómo correrlo, enlaces
Documentación técnica completa	`docs/DOCUMENTACIÓN_ServiDental.md`	Este documento
Guías por tema	`docs/*.md`	Una guía por tema (pagos, e-commerce, marcas)
Pendientes y diagnósticos	`backlog/*.md`	Planes y análisis por resolver
Decisiones	`docs/adr/` (recomendado)	Un ADR por archivo

12.2 Reglas de redacción

- **Un archivo, un tema.** Título `#` único por archivo; jerarquía `##` → `###` sin saltos.
- **Para quien llega frío:** definir siglas la primera vez; no asumir contexto.
- **Primero lo que se hace, luego el porqué.** Pasos numerados y comandos copiables.
- **Verbos en imperativo** en los procedimientos ("Ejecuta", "Configura").
- **Frases cortas.** Listas y tablas para comparar; prosa para explicar.
- **Fuente única de verdad:** no copiar contenido entre documentos; enlazar.

12.3 Formato

- Bloques de código con lenguaje (` ```bash `, ` ```ts `).
- Rutas, comandos, variables y nombres de archivo en `` `código en línea` ``.
- Enlaces relativos entre documentos del repositorio.
- Casillas `- [ ]` para checklists.
- Avisos con citas: `> **Nota:**`, `> **Advertencia:**`.
- Diagramas simples en texto o Mermaid cuando aporten.
- **Nunca** incluir secretos, tokens ni datos personales en la documentación; usar `[valor]` o `ck_xxxxx`.

12.4 Mantenimiento

- La documentación se actualiza **en el mismo PR** que el cambio de código.
- Incluir en la plantilla de PR: "¿Requiere actualizar documentación?".
- Revisión trimestral: eliminar lo obsoleto (por ejemplo, referencias a ONVO) y verificar que los pasos de arranque siguen funcionando.

- Los documentos se fechan o versionan en el encabezado cuando se entregan externamente.

12.5 Plantilla para nuevas guías

```
# Título del tema

> Una frase que resume para qué sirve este documento.

## Contexto
## Requisitos previos
## Pasos
1. …
## Verificación
## Problemas comunes
## Referencias
```

---

13. Deuda técnica y riesgos conocidos

Estos puntos deben atenderse o, como mínimo, conocerse antes de entregar el proyecto.

#	Riesgo	Severidad	Recomendación
1	**Credenciales de TiloPay con valores por defecto escritos en el código** en `src/app/tilopay-payment/page.tsx` (usuario, contraseña y llave) y expuestas al navegador mediante `NEXT_PUBLIC_TILOPAY_*`.	Alta	Eliminar los valores por defecto del código, **rotar** las credenciales en el panel de TiloPay y mover el flujo al servidor (`/api/tilopay/*`).
2	`src/utils/woocommerce.ts` lee `NEXT_PUBLIC_WOOCOMMERCE_CONSUMER_KEY/SECRET` ; si esas variables existen, la clave de WooCommerce llega al navegador.	Alta	Usar solo las variables de servidor (`WOOCOMMERCE_CONSUMER_*`) vía `/api/woocommerce/*` y retirar las `NEXT_PUBLIC_*` de Vercel.
3	`typescript.ignoreBuildErrors: true` en `next.config.js` : errores de tipos no detienen el build.	Media	Corregir errores y quitar la bandera; entretanto, ejecutar `npx tsc --noEmit` en cada PR.
4	`.env.example` no coincide del todo con el código (`TILOPAY_SECRET` vs. `TILOPAY_API_USER/PASS/KEY` , `NEXT_PUBLIC_TILOPAY_*`).	Media	Alinear el ejemplo con las variables realmente usadas.
5	`CLAUDE.md` y `README.md` desactualizados (ONVO, versión de Next, README genérico de `create-next-app`).	Media	Reescribir `README.md` con base en este documento; actualizar `CLAUDE.md` .
6	Componentes de producto duplicados (`components/products/` y `components/ecommerce/product/`).	Media	Consolidar en `components/ecommerce/product/` .
7	Dos clientes WooCommerce (`@woocommerce/woocommerce-rest-api` y `woocommerce-api`) más `server/wc.ts` .	Baja	Unificar en un solo cliente.

#	Riesgo	Severidad	Recomendación
8	Sin pruebas automatizadas ni integración continua.	Media	Agregar pruebas de utilidades críticas (`promo` , `currency` , `stock`) y un workflow de CI con lint + `tsc` + build.
9	Raíz del proyecto con archivos sueltos (`generate-types*.js` , `migrate-products*.js` , `temp/` , `FIXES_ERROR_402.md`).	Baja	Mover a `scripts/` o `docs/` , o eliminar lo obsoleto.
10	Rutas de depuración en producción (`/debug-brands` , `/api/test-woocommerce` , `/api/tilopay/test-auth`).	Media	Protegerlas o eliminarlas en producción.
11	Sin configuración de ESLint explícita en el repositorio.	Baja	Agregar configuración y reglas del equipo.

---

14. Resolución de problemas frecuentes

Síntoma	Causa probable	Solución
`API 500` en `/api/woocommerce/*`	Variables de WooCommerce ausentes o incorrectas	Revisar `WOOCOMMERCE_URL` , key y secret en el entorno
`502 Bad Gateway` en la tienda	WooCommerce caído o lento (timeout)	Verificar el estado de `wp.servidentalcr.com`
Error 402 en imágenes (Vercel)	Límite de optimización excedido	Ya mitigado con `images.unoptimized: true` ; no revertir
`site.webmanifest` o favicons con 401 en Preview	Protección de Preview de Vercel	Mantener `middleware.ts` y su `matcher` ; revisar la protección del proyecto
SDK de TiloPay con 404 / DNS	SDK cargado globalmente o URL errónea	Cargarlo solo en `/checkout` con `https://app.tilopay.com/sdk/v1/sdk.min.js`
Blog no carga	WordPress caído o `WORDPRESS_*` mal configuradas	Probar `…/wp-json/wp/v2/posts` en el navegador
Favicon u OG image antiguos tras el cambio	Caché del navegador / de redes sociales	Recarga forzada; reescanear la URL en el depurador de la red social
Promo no aparece	Etiqueta de WooCommerce distinta o fecha fuera de ventana	Verificar el slug de la etiqueta y `start` / `end` en `PROMOS`
El carrito no persiste	`CartProvider` fuera del árbol o almacenamiento bloqueado	Confirmar que envuelve la app en `layout.tsx`

Documento mantenido junto al código. Cualquier cambio de arquitectura, dependencias, variables de entorno o proceso de despliegue debe reflejarse aquí en el mismo PR.